

X5 — El cerebro de Alinvesting

Surrogate model para ajuste dinámico de parámetros

Guía técnica y conceptual (con peras y manzanas)

Proyecto Alinvesting

July 18, 2026

Este documento fusiona `x5_plan.md` y `x5_plan_redes_neuronales.md`. Explica *qué* hace X5, *cómo* está construido (LightGBM y FT-Transformer), *cómo* se entrena desde cero con backtesting paralelo y *cómo* recomienda los parámetros de trading. Pensado para leerse impreso.

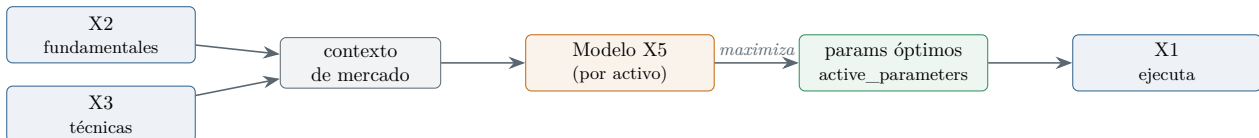
Contents

1	Qué es X5 y por qué es el cerebro	2
2	La analogía del chef	2
3	Qué entra y qué sale del modelo	2
3.1	Inputs (~240 features, vector de tamaño fijo)	2
3.2	Outputs (3 objetivos que el modelo predice)	3
4	Arquitectura que escala con los datos	3
5	El store: el cuaderno de recetas	3
6	V1 — LightGBM: el comité de expertos	3
7	V2 — FT-Transformer: atención sobre features	4
7.1	Paso 1 — Feature Tokenizer: cada número se vuelve un vector	4
7.2	Paso 2 — Atención: las features “se hablan entre sí”	4
7.3	Multi-head de salida: predecir 3 cosas a la vez	4
8	Inferencia: encontrar los parámetros óptimos	5
9	El sesgo de selección y la exploración	5
10	Cómo se ejecuta desde cero (recolección paralela)	5
11	Glosario esencial	6

1 Qué es X5 y por qué es el cerebro

Alinvesting coloca órdenes de compra en soportes calculados por X0 y las gestiona con X1. Esos módulos dependen de **parámetros de configuración** (K, N_EXP, LAMBDA, n_sizes_ejecucion, LOTAJES_M, A, B, PERDIDA_MAX). Hoy son fijos. El problema: los parámetros óptimos **cambian con el régimen de mercado** (una caída fuerte de BTC pide menos exposición que un mercado lateral).

X5 es un *surrogate model*: un modelo que predice el **retorno esperado** de una operación dado (1) el contexto de mercado y (2) los parámetros usados. Con esa predicción, en inferencia **invierte la pregunta**: “con el mercado de hoy fijo, ¿qué parámetros maximizan el retorno predicho?”. Ese set se escribe en `config/active_parameters.json`, que X1 puede leer.



Todo es por activo. X5 entrena un modelo independiente para BTCUSD, ETHUSD, TSLA, GOOGL, NVDA y AMZN. Cada uno tiene sus propios datos, su propio modelo y su propia “fase”.

2 La analogía del chef

Imagina un chef que cocinó miles de platos (operaciones) en cocinas de distintas condiciones (mercados). Lleva un cuaderno: qué ingredientes usó (parámetros), cómo estaba la cocina (contexto), y cómo quedó el plato (retorno). Con suficiente experiencia, puede predecir: “con esta receta en una cocina así, el plato sale 8/10”.

- **El chef** = el modelo de X5.
- **El cuaderno de recetas** = el *store* de trades (`resources/x5/{ACTIVO}_store.csv`).
- **Una receta** = un set de parámetros.
- **Preguntar la mejor receta para hoy** = inferencia (`--infer`).

Al principio el chef **no sabe nada**: hay que hacerlo cocinar mucho (backtesting) para llenar el cuaderno, y recién entonces puede estudiar y recomendar.

3 Qué entra y qué sale del modelo

3.1 Inputs (~240 features, vector de tamaño fijo)

Cada fila del cuaderno es una operación. Sus columnas:

- **Contexto X2+X3 en dos momentos**: al colocar el buy limit (**OE**) y al ejecutarse la orden (**OA**). Entre ambos pueden pasar días, y el mercado cambia.
- **Parámetros de config** activos en esa operación (los “ingredientes”).
- **Features temporales**: hora, día de semana, mes, proximidad a festivos US.
- **Estado del portfolio**: cuántas posiciones abiertas, exposición, retorno flotante.

El contexto de **cierre (OC)** **NO entra** al modelo: es información del futuro, no disponible en inferencia.

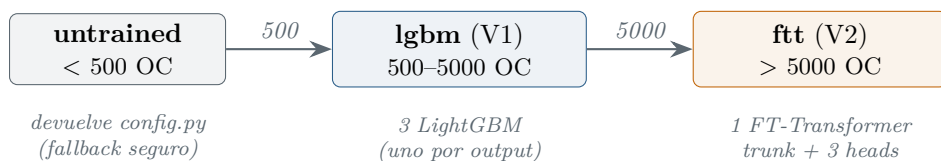
3.2 Outputs (3 objetivos que el modelo predice)

Nombre en el código	Qué mide	En qué filas
<code>retorno_pct</code>	retorno % de la operación cerrada	<code>oc</code>
<code>pnl_flotante_activo</code>	P&L flotante de posiciones abiertas	<code>oc + periodico</code>
<code>pnl_cerrado_activo</code>	P&L cerrado acumulado del activo	<code>oc</code>

El objetivo que se maximiza en inferencia es configurable; por defecto `retorno_pct` (la señal más limpia).

4 Arquitectura que escala con los datos

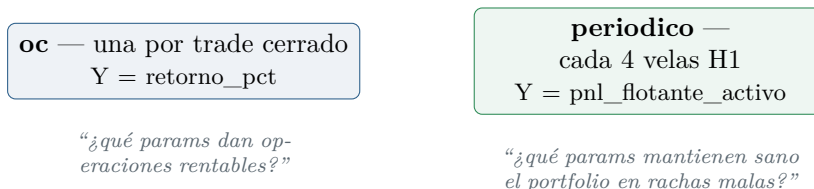
X5 no arranca con una arquitectura fija: **sube de nivel según cuántos datos tenga cada activo**. La transición es automática y por activo (umbrales en `config.py`).



Estado actual: el store está vacío, así que los 6 activos están en **untrained**. El objetivo de la Fase 1 es recolectar datos para pasar a **lgbm**.

5 El store: el cuaderno de recetas

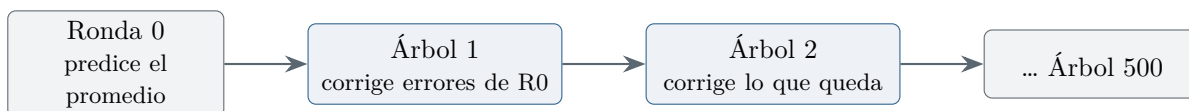
El store tiene **dos tipos de fila**, distinguidas por `tipo_registro`:



Por qué los dos: si solo se guardan filas al cerrar órdenes, durante una caída sostenida (posiciones en rojo, ninguna cierra) el modelo no recibe señal justo cuando más importa. Los registros periódicos capturan ese deterioro aunque no haya OC.

6 V1 — LightGBM: el comité de expertos

Gradient Boosting entrena muchos árboles de decisión en secuencia, donde cada árbol corrige los errores del anterior.



$$\text{retorno_predicho} = \text{promedio} + \sum_{i=1}^{500} \text{corrección}_i$$

Cada árbol es simple (“si $\text{RSI} < 50$ corrige $+1.5\%$ ”). La potencia viene de acumular cientos de correcciones pequeñas. Es robusto con pocos datos ($<50k$ filas), no requiere normalización y es interpretable. Su contra: **no es diferenciable**, así que la búsqueda de params en inferencia usa Optuna (más lenta que gradient ascent).

7 V2 — FT-Transformer: atención sobre features

Cuando el store crece (>5000 OC), el salto natural es el **FT-Transformer** (Feature Tokenizer + Transformer).

7.1 Paso 1 — Feature Tokenizer: cada número se vuelve un vector

En una red común, cada feature entra como un número. El tokenizer la convierte en un **vector** (embedding) propio y aprendido:

$$\text{embedding}_i = \text{valor}_i \times W_i + b_i$$

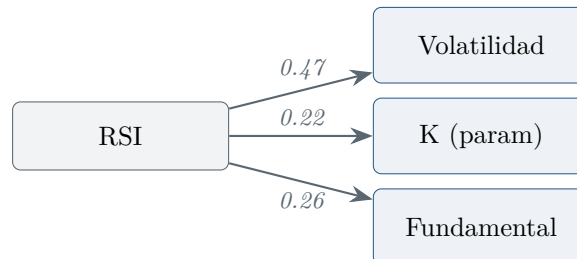


Así “RSI = 72” *significa* algo distinto que “RSI = 30”, y esa diferencia queda codificada en el vector. Cada feature tiene sus propios W_i, b_i aprendidos.

7.2 Paso 2 — Atención: las features “se hablan entre sí”

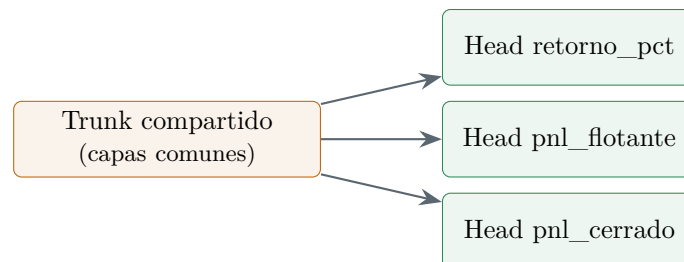
La reunión de equipo. Cada feature es una persona. En una red normal, cada una escribe su conclusión y la manda a la salida sin escuchar a las demás. En un Transformer, **antes de concluir, cada persona escucha a las otras y ajusta lo que dirá**. RSI puede pensar: “soy alto, normalmente mala señal — pero oí que la volatilidad también es alta y el fundamental es sano; quizás no sea tan malo”.

El mecanismo usa tres vectores por feature: **Q** (qué busco), **K** (qué ofrezco), **V** (lo que comparto). RSI mide su afinidad con cada compañera con el producto $Q_{RSI} \cdot K_j$, normaliza con *softmax*, y arma su nuevo embedding como suma ponderada de los V_j :



Multi-head: el mecanismo corre varias veces en paralelo (cabezas), cada una captura un tipo de relación (técnicas entre sí, params×contexto, riesgo). Se apilan varias capas: la capa 1 ve relaciones directas, la 2 relaciones entre representaciones ya contextualizadas, etc. Para datos tabulares, 2–4 capas bastan.

7.3 Multi-head de salida: predecir 3 cosas a la vez



Como un médico que hace una revisión (el trunk) y da tres diagnósticos (los heads). Compartir el trunk actúa como regularización: el modelo no puede sobreajustarse a una sola señal ruidosa si debe explicar tres a la vez.

8 Inferencia: encontrar los parámetros óptimos

Con el contexto de hoy **fijo**, se buscan los params que maximizan el retorno predicho.

- **LightGBM (no diferenciable) → Optuna**: un optimizador bayesiano que aprende qué zonas del espacio de params son prometedoras y concentra ahí sus intentos (~200 trials). Es como pedirle al comité que evalúe 200 recetas y elija la mejor.
- **FT-Transformer (diferenciable) → gradient ascent**: se calcula “¿en qué dirección mover los params sube el retorno?” y se avanza en esa dirección, con el contexto fijo. Como subir una colina con los ojos vendados tanteando la pendiente. Se repite desde varios puntos de inicio para no quedar en un máximo local.

Airbag: independiente del modelo, si el precio cayó más de un umbral (8% crypto, 5% acciones) en las últimas 4 velas H1, se fuerzan `LOTAJES_M` y `n_sizes` al mínimo. Es el último recurso; la idea es que el modelo aprenda a ser conservador antes de que el airbag se dispare.

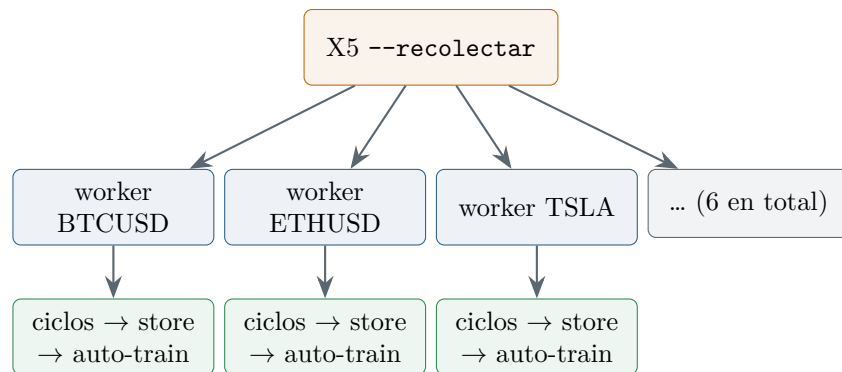
9 El sesgo de selección y la exploración

El problema más sutil. El cuaderno solo contiene operaciones hechas con los params que el sistema creía “buenos”. Nunca vemos qué habría pasado con otros. Es como juzgar cuál es el mejor restaurante yendo siempre al mismo: al final tienes 200 reseñas de uno y cero del resto.

Solución — explorar por ciclo entero. En el backtesting, un ~30% de los ciclos usan params aleatorios (EXPLORE) en vez del mejor conocido (EXPLOIT). Los ciclos EXPLORE rinden peor a propósito: su valor es enseñarle al modelo *qué no funciona* en cada régimen. Se aleatoriza por **ciclo completo** (no vela a vela) para que el modelo pueda atribuir el resultado al contexto sin ruido.

10 Cómo se ejecuta desde cero (recolección paralela)

Con el store vacío, el paso principal es `--recolectar`: lanza **un worker por activo en paralelo**; cada uno corre ciclos de backtesting, y al cruzar 500 OC entrena su propio modelo.



Cada worker usa una carpeta de trabajo aislada (`resources_{V}_{ACTIVO}`) para no chocar con los demás. Cuando termina un ciclo, **reinicia desde la fecha inicial** con otros params. Comandos:

```
# 1. estado actual (al inicio: 0 OC, todos untrained)
python scripts/X5_macro_brain.py --status

# 2. generar datos + entrenar (paralelo, 6 activos a la vez)
python scripts/X5_macro_brain.py --recolectar --version V1

# 3. recomendar params -> config/active_parameters.json
```

```
python scripts/X5_macro_brain.py --infer
#      (--infer --train reentrena antes; --infer --activo BTCUSD solo uno)

# 4. cuando confies en las metricas, activar Fase 2 en config.py:
#      TIPO_EJECUCION = "din"
```

Fases. En **Fase 1** (TIPO_EJECUCION="est") X1 usa `config.py` y X5 solo genera datos y recomienda. En **Fase 2** ("din") X1 lee `active_parameters.json` por activo; los que sigan `untrained` caen back a `config.py` automáticamente.

11 Glosario esencial

Surrogate model Modelo que imita el sistema real para predecir barato el resultado de una operación.

Embedding Representar un valor como un vector, para que el modelo lo procese con más riqueza.

Atención (Q/K/V) Mecanismo que deja que cada feature ajuste su representación mirando a las demás.

Gradient boosting Muchos árboles en secuencia; cada uno corrige el error del anterior. LightGBM lo implementa.

Gradient ascent Maximizar una función avanzando en la dirección de su gradiente. Usado en inferencia con FTT.

Overfitting El modelo memoriza el pasado en vez de aprender patrones generales.

OE / OA / OC Orden en Espera / Orden Abierta / Orden Cerrada.

model_status Estado por activo en el JSON: `untrained` / `lgbm` / `ftt`.